

基于深度学习的 NIfTI 图像分割与工程化部署

nnU-Net v2、MONAI SegResNet 与 EfficientNet-B0 U-Net 的设计、训练、评估及 WebUI 实现

13

NIfTI cases

3

model families

0.9555

best test Dice

摘要

本项目面向 NIfTI 三维扫描数据的二值语义分割任务，目标是在统一数据划分、评价指标与推理接口下，对比 nnU-Net v2、MONAI SegResNet 和以 EfficientNet-B0 为编码器的 2D U-Net 三条技术路线，并提供可供验收使用的批量推理 WebUI。项目对 13 个 case 完成目录规范化、标签二值化、几何一致性校验和 case-level 划分，避免同一个体的切片跨训练、验证和测试集合造成信息泄漏。所有输出 mask 均保留 NIfTI affine/header，并以 `.nii.gz` 保存。

nnU-Net v2、MONAI SegResNet 和 EfficientNet-B0 U-Net 的测试宏平均 Dice 分别为 0.9490、0.9555 和 0.9065；前两者在共同测试 case `2_25_xy` 与 `s_3` 上评估，均超过课程提出的 0.92 目标。SegResNet 以 4.70M 参数取得最高 Dice、二类 mIoU 0.9269 和 Recall 0.9650，说明 3D 上下文对连续体数据有效；nnU-Net 的 Accuracy 0.9587 与 Precision 0.9581 略占优势，体现更保守的前景判定。

本项目代码已经开源在 <https://github.com/Jerry050512/img-seg>。

关键词：NIfTI；二值语义分割；nnU-Net v2；SegResNet；EfficientNet-B0；case-level split；WebUI

目录

项目分工	4
1 实验目标与任务定义	4
1.1 实验背景	4
1.2 项目目标	4
2 数据集、标注与数据制备	4
2.1 数据集概况	4
2.2 标注流程与标签规范	5
2.3 数据增强策略	5
2.4 制备经验与不足	5
3 算法原理与统一分割框架	5
3.1 像素级二分类	5
3.2 Encoder-Decoder 与 Skip Connection	5
4 评价指标与计算方式	6
4.1 混淆矩阵	6
4.2 分割性能指标	6
4.3 模型复杂度与效率	6
5 三种模型架构与参数设置	7
5.1 nnU-Net v2 2D	7
5.2 MONAI SegResNet	7
5.3 EfficientNet-B0 U-Net	8
5.4 架构差异总结	8
6 网络训练、推理与 WebUI	8
6.1 实验环境	8
6.2 训练过程	9
6.3 统一推理接口与 WebUI	9
7 实验结果与对比分析	11
7.1 训练收敛	11
7.2 验证、测试与综合分析	12
8 项目代码文件与功能介绍	17
8.1 课程交付入口文件	17
8.2 核心源码模块	17
8.3 配置、工具与测试	17
9 个人理解、实验体会与总结	18
9.1 对深度学习分割的理解	18
9.2 工程实践体会	18
9.3 实验结论	18
10 参考文献	19
11 附录：运行命令与交付清单	20
11.1 环境与检查	20
11.2 nnU-Net 训练、推理与评价	20
11.3 SegResNet 训练、推理与评价	20
11.4 EfficientNet-B0 训练、推理与评价	20
11.5 报告构建	20

项目分工

学号	姓名/账号	主要负责内容	工作量
[待补充]	@Jerry050512	nnU-Net v2 全流程、统一推理链路与 WebUI 开发；课程报告整合	40%
[待补充]	@flypigff	MONAI SegResNet 的 3D patch 训练、推理与结果分析	30%
[待补充]	@NH-5	EfficientNet-B0 编码器的 2D U-Net 训练、推理与结果分析	30%

表 1 项目成员分工与工作量声明

1 实验目标与任务定义

1.1 实验背景

语义分割要求模型为输入中的每个像素或体素分配类别。与整图分类相比，分割不仅要判断目标是否存在，还要恢复其空间范围、孔洞结构和边界。本项目数据以 NIfTI 体数据形式存储，单个 case 由多张连续切片组成；输出为与输入几何信息一致的二值 mask。任务难点包括样本量小、不同 case 的平面尺寸和切片数量差异明显、目标内部存在大量规则或不规则孔洞，以及大尺寸 case 对显存和主存提出较高要求。

U-Net 通过对称的 Encoder-Decoder 和跳跃连接兼顾语义与定位信息，是医学图像分割的经典基础架构 [1]。nnU-Net 在此基础上根据数据 fingerprint 自动规划预处理、patch、batch size 和网络深度，强调“由数据决定配置” [2]。本项目同时选择可控的 3D SegResNet 与轻量 2D EfficientNet-B0 U-Net，以观察自配置强基线、三维上下文模型和二维轻量模型之间的精度与效率取舍。

1.2 项目目标

- 建立统一、可复现的数据读取、标签处理和 case-level 划分流程；
- 在相同数据协议下训练 nnU-Net v2、MONAI SegResNet 和 EfficientNet-B0 U-Net；
- 统一计算 mIoU、Dice、Accuracy、Precision、Recall、参数量、FLOPs 和推理速度；
- 通过收敛曲线、定量表格、雷达图和典型切片进行多角度分析；
- 暴露与模型内部实现解耦的批量推理接口，并以 Gradio WebUI 支持模型选择、输入/输出目录、进度显示和二值 mask 输出；
- 形成包含可运行命令、实验限制与后续改进方向的完整课程报告。

2 数据集、标注与数据制备

2.1 数据集概况

规范化后的本地数据集包含 13 个 case，每个 case 仅保留一对 `image.nii.gz` 与 `mask.nii.gz`。图像平面大小从 512×512 到约 1319×914 不等，切片数从 156 到 351 不等；前景比例从 9.9428% 到 46.0658%，表明不同样本的目标规模差异较大。所有样本当前 spacing 均记录为 $1.0 \times 1.0 \times 1.0$ 。

样本组	shape 范围	case 数	前景比例	用途
全部数据	$512 \times 512 \times 156$ 至 $1319 \times 914 \times 245$ 等	13	9.94%-46.07%	统一数据池
Train	多尺寸、多切片	9	见审计文档	参数学习
nnU-Net Val	1_23_XY; nose_layer12	2	32.90%、44.37%	checkpoint 选择
SegResNet Val	1_23_XY; nose_layer4	2	32.90%、29.75%	checkpoint 选择
nnU/Seg Test	2_25_XY; S_3	2	40.97%、42.76%	独立评估
EfficientNet Test	2_25_XY; S_4	2	40.97%、46.07%	分支基线

表 2 数据集规模、划分与用途概览

三条模型线均使用随机种子 42，并按 case 形成 train=9、validation=2、test=2。因此本文可直接比较 nnU-Net 与 SegResNet 的测试结果，但验证曲线只用于各自 checkpoint 选择；EfficientNet 作为非严格基线。

2.2 标注流程与标签规范

课程标注实践使用 ITK-SNAP [3] 的 Polygon 工具，对 `nose_layer4` 逐层手工勾画目标 mask；该样本属于本文当前 13-case 数据集，并用于 SegResNet 验证。为避免多人重复编辑同一切片，三位成员按 z 轴连续区间分工：z=63-94、z=95-124 和 z=125-156。具体流程为：

1. 在 ITK-SNAP 中载入原始 NIfTI 体数据，确认轴向方向和切片编号；
2. 按约定的 z 轴范围逐层浏览，用 Polygon 工具沿目标外轮廓和内部孔洞建立封闭区域；
3. 将目标区域写入二值 mask，背景保持为 0，并在区间交界切片附近检查轮廓是否连续；
4. 导出 NIfTI mask，核对 image/mask 的 shape 与 affine，确保空间位置一致；
5. 对二分类标签统一执行 `mask > 0`，以 `uint8` 的 `{0, 1}` 保存，同时保留原始 header；
6. 运行数据审计，检查标签集合、前景比例和重复文件；无法自动判断的冲突只记录，不覆盖原始数据。

2.3 数据增强策略

nnU-Net 在训练时自动组合空间和强度增强。空间增强包括 $\pm 15^\circ$ 随机旋转、0.7-1.4 随机缩放和双轴镜像；强度增强包括高斯噪声、高斯模糊、亮度乘法、对比度变化、低分辨率模拟以及 gamma 变换。增强只施加于训练 patch，验证与测试保持确定性预处理。前景 patch 过采样比例为 0.33，以缓解大面积背景主导梯度的问题。EfficientNet-B0 U-Net 将 z 轴切片缩放至 512×512 ，保留全部前景切片并随机保留 25% 空 mask；验证和测试保留每个 case 的全部 slice。MONAI SegResNet 先对非零体素做强度标准化，再按前景/背景 1:1 随机采样 $96 \times 96 \times 64$ ROI；空间增强包括三个轴向的随机翻转和 90° 旋转，强度增强包括随机尺度与偏移。每个训练体每轮采样 2 个 patch，验证与测试使用 `overlap=0.5` 的高斯加权滑窗，均不采用随机增强。

2.4 制备经验与不足

本次数据制备最重要的经验是数据几何和划分规则比模型调参更应优先固定。NIfTI 的数组 shape 相同并不代表物理空间必然一致，因此 affine/header 校验不可省略；按 slice 随机划分会把同一个体的相邻切片泄漏到不同集合，导致异常乐观的测试结果；二值任务若不统一非零标签，会使损失函数错误解释类别。不足主要有三点。第一，仅 13 个 case 且验证/测试各 2 例，宏平均对单例差异高度敏感。第二，所有 spacing 均为 1.0，需确认这是设备真实物理间距还是导出时的默认值。第三，原始标注流程和一致性复核记录不完整。后续应补充标注元数据、采用 5-fold case-level 交叉验证，并报告均值与标准差。

3 算法原理与统一分割框架

3.1 像素级二分类

设输入切片或体数据为 x ，模型输出目标类概率 $p = f_{\theta(x)}$ 。阈值化得到预测 $\hat{y} = 1[p \geq 0.5]$ 。训练的核心是在保持目标区域重叠的同时约束逐像素分类。Dice loss 直接优化区域重叠，交叉熵或 BCE 提供稳定的像素级梯度；二者结合可兼顾类别不平衡与局部分类。nnU-Net 使用 Dice 与 Cross Entropy 的组合损失，并在解码器多尺度输出上进行 deep supervision。EfficientNet-B0 U-Net 与 MONAI SegResNet 的项目配置为 Dice+BCE。三者最终都输出二值 mask，并通过同一评价模块转换为布尔数组后计算混淆矩阵。

3.2 Encoder-Decoder 与 Skip Connection

编码器逐级下采样，把局部纹理压缩为高层语义；解码器逐级上采样恢复空间分辨率。跳跃连接把编码器同尺度特征直接传给解码器，弥补下采样造成的位置细节损失。对本数据中的细孔、网格和不规则外轮廓而言，高分辨率浅层特征对于边界定位尤其重要。2D 模型将每个体数据视为切片序列，显存开销较低且易于使用 ImageNet 预训练，但无法直接观察相邻切片连续性。3D 模型以体 patch 进行卷积，能利用跨层上下文，代价是显存占用和计算量明显增大。nnU-Net 的 2D 配置选择是本机 8 GB 显存和大平面尺寸下的实际折中。

4 评价指标与计算方式

4.1 混淆矩阵

将目标类视为正类：TP 为正确预测的前景体素，FP 为误预测前景，FN 为漏掉的前景，TN 为正确背景。所有区域指标都从这四项计算。项目按每个 case 独立计算，再对 case 做宏平均，使体积较大的样本不会完全支配最终结果。

4.2 分割性能指标

指标	公式	解释
IoU	$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c}$	预测与标注交集占并集的比例
mIoU	$\text{mIoU} = \frac{1}{C} \sum_{c=1}^C \text{IoU}_c$	对类别 IoU 取平均；本文二类含前景与背景
Dice/F1	$\text{Dice} = \frac{2 \text{TP}}{2 \text{TP} + \text{FP} + \text{FN}}$	对目标重叠敏感，是核心指标
Accuracy	$\text{Acc} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$	整体像素正确率，背景多时可能偏乐观
Precision	$P = \frac{\text{TP}}{\text{TP} + \text{FP}}$	预测为目标的像素中有多少正确
Recall	$R = \frac{\text{TP}}{\text{TP} + \text{FN}}$	真实目标中有多少被找出

表 3 二值分割评价指标定义与计算公式 [4], [5]

项目现有 `metrics.json` 中字段 `iou` 指前景 IoU。为严格满足课程的 mIoU 要求，本文利用保留的 TP/FP/FN/TN 另算背景 IoU，并取二者平均。最佳 nnU-Net 的宏平均前景 IoU 为 0.9048、二类 mIoU 为 0.9192；SegResNet 对应数值为 0.9157 和 0.9269。EfficientNet 分支未保留原始混淆矩阵，本文依据其四舍五入后的逐 case Precision/Recall 和审计前景比例近似恢复 Accuracy 0.9233 与 mIoU 0.8543，相关值只用于补充展示，并在原始 JSON 中标注为推导值。

4.3 模型复杂度与效率

参数量为所有可学习张量元素数之和：

$$\text{Params} = \sum_{l=1}^L |\theta_l|$$

卷积层理论计算量与输出空间尺寸、输入/输出通道及卷积核大小相关。对普通二维卷积，若乘法和加法分别算一次 FLOP，则近似为：

$$\text{FLOPs}_{\text{conv}} = 2H_{\text{out}}W_{\text{out}}C_{\text{out}} \frac{C_{\text{in}}}{g} K_h K_w$$

其中 g 为 groups。本文使用 forward hook 对 512×512 单通道输入逐层统计卷积和转置卷积；不含归一化、激活、内存搬运与预处理，因此属于理论主干计算量，不能替代真实延迟。参数量、计算量与运行环境应和精度一起报告，避免仅凭单一指标判断模型优劣 [6]。

推理效率定义为：

$$\text{FPS} = \frac{N}{T}, \quad \text{latency} = \frac{T}{N} \times 1000 \text{ ms}$$

对于体数据，报告同时给出 sec/case。将整例耗时除以切片数得到的 ms/slice 包含 NIFTI 读取、预处理、网络、重采样与导出，是端到端摊销速度；它与只测网络 forward 的 micro-benchmark 不应直接混比。

5 三种模型架构与参数设置

5.1 nnU-Net v2 2D

nnU-Net 会从数据 fingerprint 推导 spacing、patch、batch size、网络拓扑和标准化策略 [2]。本实验实际网络为 8 stage PlainConvUNet：编码器通道数依次为 32、64、128、256、512、512、512、512；每 stage 含两个 3×3 Conv2d，步幅从第二 stage 起为 2×2 ；每层采用 InstanceNorm2d 和 LeakyReLU。解码器通过转置卷积上采样、拼接跳跃特征，并在 7 个尺度上使用 deep supervision。

项目	实际值	说明
配置	2D fold 0	基于 nnUnetPlans 自动规划
输入 patch	1024×640	中位平面尺寸约 910×515
Batch size	5	单卡 RTX 4060 Laptop GPU
Epoch	200	每 epoch 250 train + 50 val iteration
优化器	SGD	初始 lr 0.01, momentum 0.99, Nesterov
正则	weight decay $3e-5$	前景过采样 0.33, 在线增强
损失	Dice + CE	多尺度 deep supervision
归一化	Z-score	每通道强度标准化

表 4 nnU-Net v2 2D 训练配置

5.2 MONAI SegResNet

SegResNet 使用残差块构建 3D Encoder-Decoder，以残差连接改善深层网络的梯度传播；3D 卷积可以直接利用相邻切片上下文。项目基于 MONAI 统一组件实现此路线 [7]，其架构思想与 3D 残差分割网络相关 [8]。

实际网络输入/输出通道均为 1, 初始 filters=16; 编码端残差块数为 1/2/2/4, 解码端为 1/1/1, 使用 InstanceNorm 和 dropout=0.1。训练采用 ROI= $96 \times 96 \times 64$ 、有效 patch batch=2、200 epoch、AdamW、初始 learning rate= $2e-4$ 、余弦退火、AMP 和 Dice+BCE loss；每 10 epoch 对两个验证 case 做完整滑窗评价。epoch 170 的 [checkpoints/monai_segresnet/best.pt](#) 取得最佳验证 Dice 0.8261，测试宏平均 Dice 0.9555、mIoU 0.9269。

项目	实际值	说明
3D ROI	$96 \times 96 \times 64$	各维可被下采样倍率整除
Patch batch	2	<code>batch_size=1</code> , 每体采样 2 个 ROI
优化器	AdamW	lr= $2e-4$, weight decay= $1e-5$
训练	200 epoch / AMP	每 10 epoch 完整体验证
推理	overlap=0.5	高斯加权 sliding window, batch=4
规模	4.70M Params	单 ROI 卷积约 81.69 GFLOPs

表 5 MONAI SegResNet 训练与推理配置

5.3 EfficientNet-B0 U-Net

EfficientNet 通过复合缩放同时协调网络深度、宽度和输入分辨率 [9]。项目使用 `segmentation_models_pytorch.Unet`，以 ImageNet 预训练 EfficientNet-B0 作为编码器，解码器沿用 U-Net 跳跃连接。输入为 z 轴单通道切片，统一缩放至 512×512 ，输出 1 通道 logits。

实际配置为 100 epoch、batch size=8、AdamW、learning rate=3e-4、weight decay=0.01、AMP、Dice+BCE 和阈值 0.5；保留全部含前景切片和 25% 空切片，体推理后逐层重组 NIFTI。 `checkpoints/efficientnet_b0/20260710T152124825334Z/best.pth` 在 epoch 3 达到最佳验证 Dice 0.7916，随后训练 loss 继续下降但验证性能未再提高，表现出较早的过拟合。其测试宏平均 Dice 0.9065、前景 IoU 0.8313、Precision 0.9281、Recall 0.8878，端到端耗时 11.16 s/case。按当前依赖与配置实际构建模型后，共有 6.250893 M 参数；对单张 512×512 切片统计卷积与转置卷积，计算量为 23.5836 GFLOPs。

项目	实际值	说明
输入	512×512	z 轴单通道切片
Batch size	8	CUDA 上启用 AMP
优化器	AdamW	lr=3e-4, weight decay=0.01
损失	Dice + BCE	二值输出阈值 0.5
采样	前景 100%，空层 25%	验证/测试保留全部切片
规模	6.250893 M Params	单切片卷积约 23.5836 GFLOPs

表 6 EfficientNet-B0 U-Net 训练与推理配置

5.4 架构差异总结

模型	维度	配置方式	优势	主要风险
nnU-Net v2	2D	数据驱动自配置	强基线、流程完整	网络较重，大图显存压力
MONAI SegResNet	3D	人工配置 ROI	跨切片连续性	小样本过拟合、显存高
EfficientNet-B0 U-Net	2D	预训练轻量编码器	速度与部署友好	缺少 3D 上下文

表 7 三种分割架构的设计取舍

6 网络训练、推理与 WebUI

6.1 实验环境

三条模型线使用的硬件并不相同，因此速度数据只用于说明各自运行条件。

模型	硬件	系统	框架环境
nnU-Net v2	Core i7-13700H; 16 GB RAM; RTX 4060 Laptop 8 GB	Windows / WDDM	PyTorch 2.5.1+cu121; nnU-Net v2
MONAI SegResNet	RTX 4090 D 24 GB	Linux	PyTorch 2.6.0+cu124; MONAI 1.5.1
EfficientNet-B0 U-Net	NVIDIA GPU PC (型号未记录)	Windows	PyTorch; segmentation-models-pytorch

表 8 三条模型线的训练与推理环境

Windows 图形驱动环境下可用显存和连续内存分配并不总是一致，大尺寸 case 在模型迁移阶段仍可能触发分配失败；Linux 服务器则为 3D 滑窗推理提供了更充足的显存。

项目统一使用 `uv` 管理 Python 3.12 环境和命令。核心依赖包括 PyTorch、nnU-Net v2、MONAI、segmentation-models-pytorch、nibabel、Albumentations、Matplotlib 与 Gradio。超参数位于 `configs/<model>/`，训练脚本必须接收配置文件路径，不在代码中硬编码数据和输出路径。

6.2 训练过程

nnU-Net 完成 200/200 epoch, 墙钟训练时长约 8 h 27 min。大尺寸标签在标准预处理的前景坐标物化阶段消耗过多主存, 因此实验使用有界内存采样完成 11 个 train/validation case 的预处理。训练采用单数据增强进程, 降低 Windows shared-memory 不稳定性。

训练命令的标准入口为:

```
uv sync
uv run imgseg-nnnet --config configs/nnnet_v2/base.yaml prepare
uv run imgseg-nnnet --config configs/nnnet_v2/base.yaml plan
uv run imgseg-nnnet --config configs/nnnet_v2/base.yaml train --configuration 2d --fold 0
```

SegResNet 完成 200 epoch, 并通过 `last.pt` 两次断点恢复。日志内训练循环累计 1303.3 s、20 次完整验证累计 935.5 s, 合计可计量计算时间 2238.8 s (约 37.3 min); 该值不含三次 NIFTI cache 重建、中断等待和进程迁移, 因此不是严格墙钟总时长。训练 loss 的 10-epoch 移动平均总体下降, 验证 Dice 从 epoch 5 的 0.5938 上升至 epoch 170 的 0.8261, 此后在 0.80 左右波动, 故选择 epoch 170 而非 epoch 200。

```
uv run imgseg-segresnet --config configs/monai_segresnet/base.yaml inspect
uv run imgseg-segresnet --config configs/monai_segresnet/base.yaml train
uv run imgseg-segresnet --config configs/monai_segresnet/base.yaml predict --checkpoint checkpoints/monai_segresnet/best.pt --split test
uv run imgseg-segresnet --config configs/monai_segresnet/base.yaml evaluate --prediction-dir outputs/monai_segresnet --split test
```

EfficientNet 完成 100 epoch, ImageNet encoder 初始化成功。最佳验证 Dice 0.7916 在 epoch 3 即出现, 说明小样本切片训练很快进入验证平台; 后续应使用 early stopping 缩短训练, 并在不接触测试集的前提下调整增强和正则。

```
uv run imgseg-efficientnet train --config configs/efficientnet_b0/base.yaml
uv run imgseg-efficientnet evaluate --config configs/efficientnet_b0/base.yaml --checkpoint checkpoints/efficientnet_b0/20260710T152124825334Z/best.pth --split test
```

6.3 统一推理接口与 WebUI

WebUI 基于 Gradio [10] 构建, 不直接调用某个模型内部层, 而通过公共 inference 模块完成输入收集、模型/权重选择、推理、输出二值化、结果评价和预览。nnU-Net、SegResNet 与 EfficientNet 均已接入统一选择; SegResNet 对二维普通图片给出明确错误, 仅接受与训练协议一致的 3D NIFTI。界面已经实现:

- 选择 nnU-Net/SegResNet/EfficientNet 与对应 checkpoint;
- 输入单个 NIFTI、NIFTI 目录、单张 JPG/PNG 或图片目录;
- 填写输出目录, 默认 `Test_Seg`;
- 显示批量推理进度、单例状态与耗时, 并支持取消;
- NIFTI 输出 `.nii.gz` 二值 mask, 2D 图片输出同名 PNG;
- 可选参考 mask 时显示 Dice、IoU、Precision 和 Recall;
- 生成输入/预测预览, WebUI 仅依赖统一接口。

启动命令为:

```
uv run imgseg-webui
```

普通 2D 图片被包装为单通道单 slice 的临时 NIFTI, 使用 identity affine, 因此该输出只表达像素空间分割, 不代表真实物理坐标。课程展示时应明确这一限制。

当前 WebUI 已统一接入三个模型。公共输入收集器同时修正了 `case/image.nii.gz` 布局的 case id 推导, 使其能与同目录 `mask.nii.gz` 正确配对。

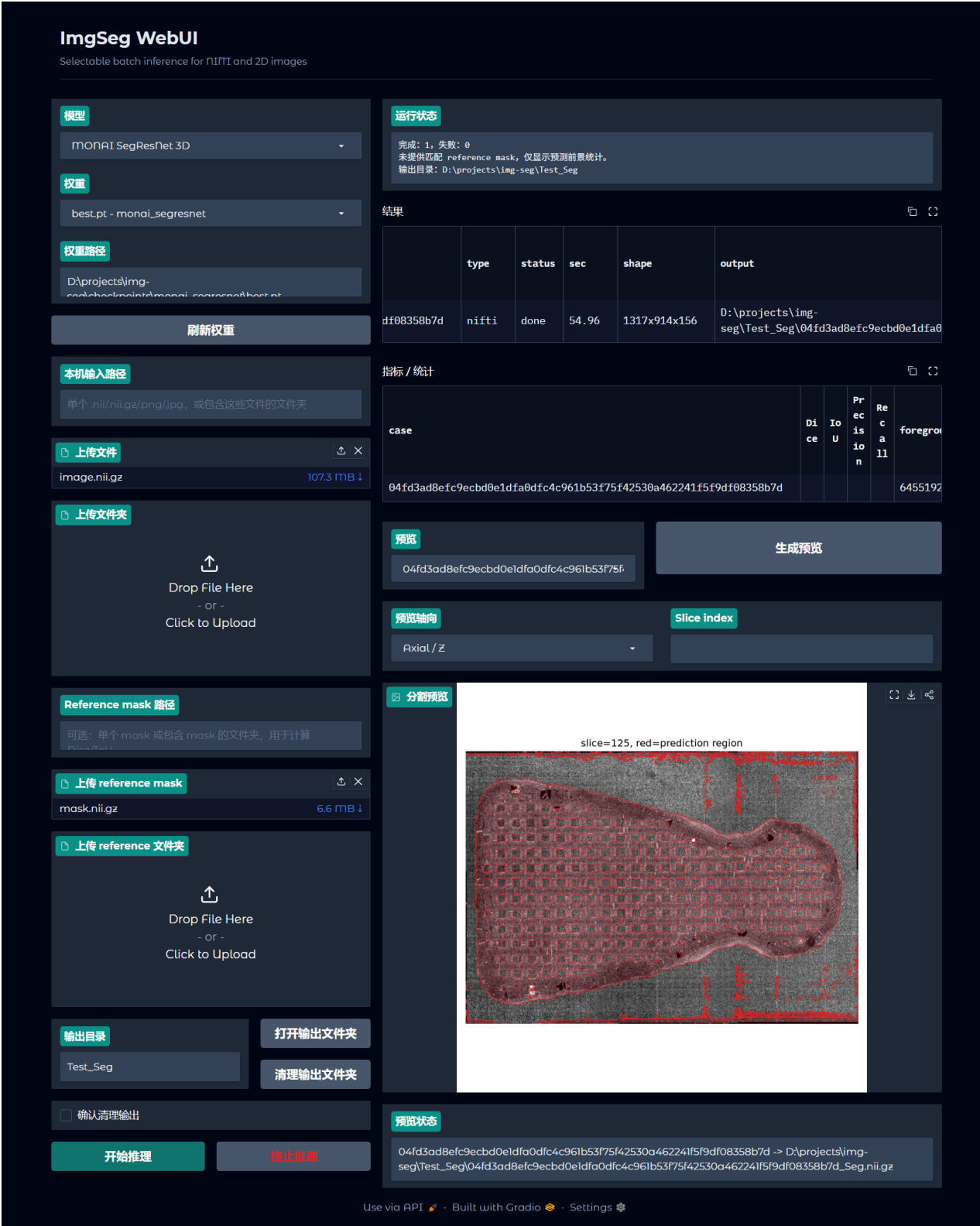


图 1 WebUI 实际运行界面：选择 MONAI SegResNet 3D 和 best.pt，上传 .nii.gz 图像与参考 mask，完成推理后展示运行状态、逐例结果、指标区域和分割预览。

7 实验结果与对比分析

7.1 训练收敛

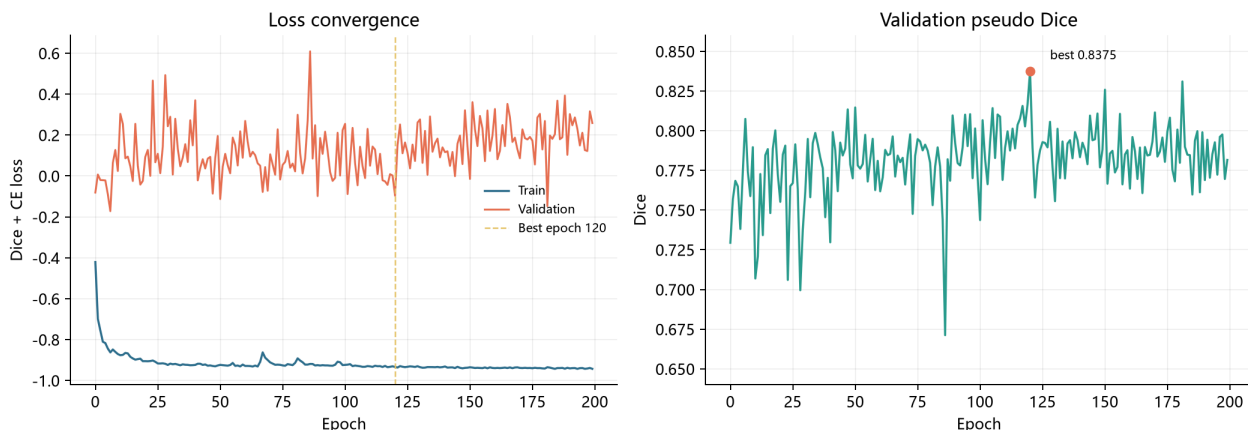


图2 nnU-Net v2 200 epoch 训练/验证 loss 与 validation pseudo Dice。最佳 pseudo Dice 0.8375 出现在 epoch 120。

如图2所示，训练 loss 总体继续下降，而 validation loss 和 pseudo Dice 存在明显波动。epoch 120 之后，训练指标仍改善但验证性能未稳定提高，说明小验证集和强增强共同造成较高方差，也提示继续训练并不等价于更好泛化。final pseudo Dice 降至 0.7816，因此 best checkpoint 比 final 更合理。

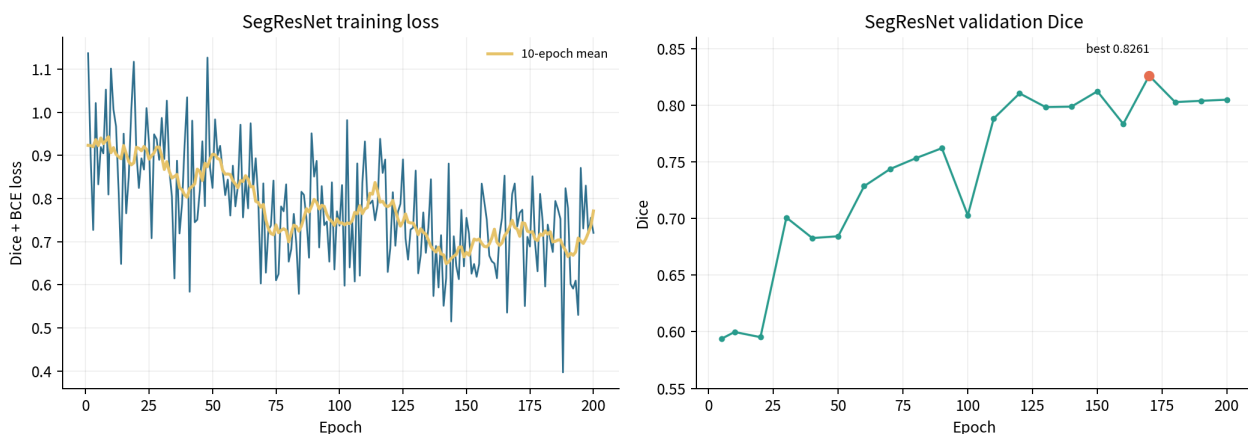


图3 SegResNet 200 epoch patch 训练 loss 与每 10 epoch 完整体验证 Dice。最佳验证 Dice 0.8261 出现在 epoch 170。

SegResNet 的单 epoch loss 因每轮仅从 9 个训练体随机抽取 18 个 patch 而明显抖动；10-epoch 移动平均仍显示从约 0.92 降至约 0.70。验证 Dice 在 epoch 110 后进入约 0.79-0.83 区间，说明继续增加 epoch 的边际收益有限。epoch 170 至 200 的回落同时证明应以验证集选 checkpoint，而不能默认采用最后一轮。

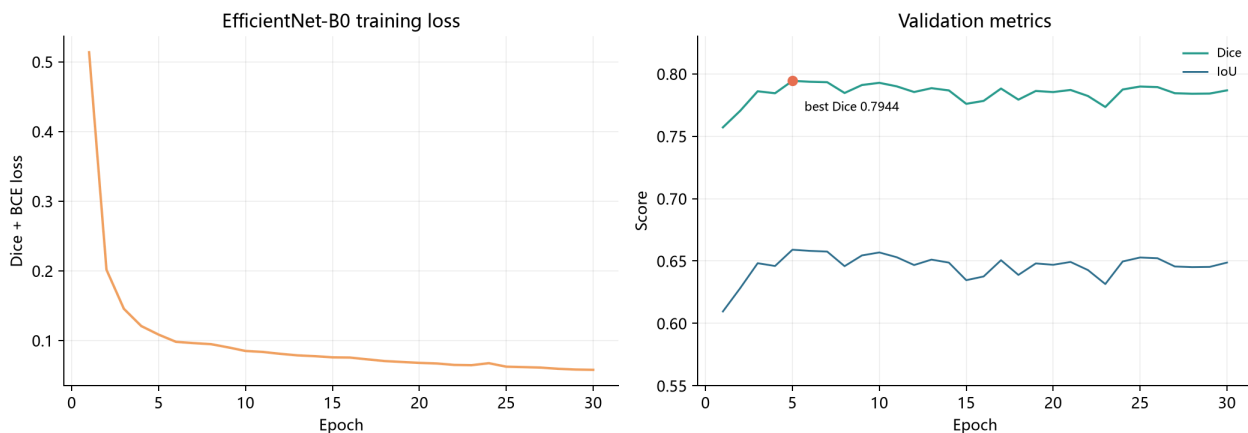


图4 最新保留的 EfficientNet-B0 30 epoch run：训练 Dice+BCE loss、验证 Dice 与 IoU。最佳验证 Dice 0.7944 出现在 epoch 5。

如图 4 所示, 训练 loss 从 0.5137 持续下降至 0.0579, 但验证 Dice 在 epoch 5 达到 0.7944 后长期维持在约 0.78-0.79, 训练与验证走势明显分离。这与 100 epoch 选定实验在早期达到最佳验证点的现象一致, 支持采用 early stopping, 避免只根据训练 loss 继续下降而延长训练。

7.2 验证、测试与综合分析

7.2.1 nnU-Net 整例验证结果

Validation case	Dice	前景 IoU	Precision	Recall
1_23_XY	0.7456	0.5944	0.8525	0.6626
nose_layer12	0.8017	0.6690	0.9121	0.7151
宏平均	0.7737	0.6317	0.8823	0.6889

整例验证 Dice 0.7737 明显低于测试 Dice 0.9490。验证集只有 2 例且包含较大平面样本, 这种差异不能简单解释为测试集“更容易”或模型“泛化极好”; 更可靠的判断需要 5-fold 交叉验证和置信区间。

7.2.2 nnU-Net 独立测试与 checkpoint 对比

Checkpoint	Dice	前景 IoU	mIoU	Accuracy	Precision	Recall
best (epoch 120)	0.9490	0.9048	0.9192	0.9587	0.9581	0.9405
final (epoch 199)	0.9417	0.8922	0.9090	0.9533	0.9551	0.9299

best 在所有主要宏平均指标上均优于 final: Dice 提高 0.0073, 二类 mIoU 提高 0.0102, Recall 提高 0.0106。差异虽然不大, 但方向一致, 支持按 validation pseudo Dice 选择 `checkpoint_best.pth`。

Test case	Dice	前景 IoU	mIoU	Accuracy	Precision	Recall
2_25_XY	0.9156	0.8444	0.8690	0.9326	0.9397	0.8927
s_3	0.9823	0.9652	0.9694	0.9848	0.9764	0.9882
宏平均	0.9490	0.9048	0.9192	0.9587	0.9581	0.9405

s_3 的各项指标接近 0.98, 而 2_25_XY Recall 仅 0.8927, 主要误差表现为漏分。两个 case 之间 Dice 相差 0.0667, 说明数据形态和尺寸仍显著影响模型表现。

7.2.3 SegResNet 验证与测试结果

Validation case	Dice	前景 IoU	Precision	Recall
1_23_XY	0.7785	0.6373	0.7701	0.7870
nose_layer4	0.8737	0.7757	0.8149	0.9416
宏平均	0.8261	0.7065	0.7925	0.8643

Test case	Dice	前景 IoU	mIoU	Accuracy	Precision	Recall
2_25_XY	0.9342	0.8765	0.8937	0.9454	0.9231	0.9455
s_3	0.9769	0.9548	0.9602	0.9801	0.9694	0.9845
宏平均	0.9555	0.9157	0.9269	0.9627	0.9462	0.9650

SegResNet 的测试 Dice 比验证 Dice 高 0.1294。该差距由两个极小集合的样本构成差异造成: 验证中的 1_23_XY 仅 0.7785, 而测试中的 s_3 达到 0.9769。它在两个测试 case 上都超过 0.93, 说明结果并非只由单例抬高; 但每组仅 2 例, 仍不足以估计稳定泛化性能。文件名族如 s_1/s_3/s_4/s_5、nose_layer* 是否来自同一对象也需要原始元数据确认, 否则 case-level split 仍可能低估 subject-level 相关性。

7.2.4 EfficientNet-B0 测试基线

Test case	Volume shape	Dice	前景 IoU	Precision	Recall	sec/case
-----------	--------------	------	--------	-----------	--------	----------

2_25_XY	973 × 973 × 298	0.8674	0.7658	0.9195	0.8208	16.25
s_4	512 × 512 × 331	0.9456	0.8968	0.9367	0.9547	6.06
宏平均	—	0.9065	0.8313	0.9281	0.8878	11.16

EfficientNet 在大尺寸 2_25_XY 上 Recall 0.8208，说明逐 slice 轻量模型的主要问题仍是漏分；s_4 上 0.9456 Dice 则说明 ImageNet encoder 对规则纹理具有较强迁移能力。由于第二个测试 case 与另外两模型不同，表中均值只能用于描述该分支自身结果。

7.2.5 定量性能、复杂度与效率

模型	mIoU	Dice	Acc.	Prec.	Recall	Params/ M	FLOPs/ G	sec/case
nnU-Net v2 2D	0.9192	0.9490	0.9587	0.9581	0.9405	46.32	119.27 (a)	426.96 (c)
MONAI SegResNet	0.9269	0.9555	0.9627	0.9462	0.9650	4.70	81.69 (b)	25.60 (d)
EfficientNet-B0 U-Net	0.8543 (e)	0.9065 (f)	0.9233 (e)	0.9281 (f)	0.8878 (f)	6.25	23.58 (a)	11.16 (f)

表 9 三模型测试性能、复杂度与整例推理耗时对比

表中 (a) 为单张 512 × 512 的 2D 网络卷容量；SegResNet 的 81.69 G 为单个 96 × 96 × 64 3D ROI，二者不能直接比较。(c) 混合 2_25_XY CPU fallback 与 s_3 RTX 4060；(d) 为 RTX 4090 D；(e) 由四舍五入逐例指标和前景比例近似恢复；(f) EfficientNet 使用 s_4 而非 s_3，且 GPU 型号未记录。粗体仅表示表内数值最优，不代表严格受控实验下的显著优势。

模型/样本	端到端耗时	摊销吞吐/延迟	硬件与口径
nnU-Net / s_3	32.079 s/case	9.07 slice/s; 110.24 ms/slice	RTX 4060; 291 slices; TTA off
SegResNet / test mean	25.596 s/case	11.51 slice-equiv/s; 86.91 ms/slice-equiv	RTX 4090 D; 3D 滑窗; 仅作切片等效换算
EfficientNet / test total	11.16 s/case	28.19 slice/s; 35.47 ms/slice	GPU 型号未记录; 629 slices 加权

表 10 推理吞吐与单切片摊销延迟；包含读取、预处理、网络、重采样与保存

速度结果已经按课程要求给出 FPS 等价吞吐和单切片延迟，但三行的硬件、测试 case 与 2D/3D 推理方式不同，只能说明各自实际运行效率。FLOPs 仅统计卷积与转置卷积；严格横评应在同一 GPU、同一 test split 和相同 I/O 边界下重测。

Case	nnU-Net	SegResNet	EfficientNet	统一环境与说明
s_3	58.14 s	18.32 s	5.77 s	RTX 4060 8 GB; 三者完成
2_25_XY	CUDA OOM	120.48 s	15.15 s	RTX 4060 8 GB; nnU-Net 未完成

表 11 同机端到端复测；包含模型加载、NIfTI I/O、推理和导出

为缩小硬件差异，本文又在一台 RTX 4060 Laptop 8 GB、Windows/WDDM 环境下通过根目录 test.py 复测。s_3 上三个模型均完成，EfficientNet、SegResNet、nnU-Net 依次约为 5.77、18.32、58.14 s。大尺寸 2_25_XY 上前两者完成，但 nnU-Net 在网络迁移到 GPU 时 CUDA OOM，因此无法形成覆盖两个共同 case 的三模型完整同机排名。EfficientNet 此处使用本机最新保留的 30-epoch checkpoint，而主精度表采用 Dice 更高的 100-epoch 实验；同机表只用于工程吞吐诊断。原始口径与状态保存在 assets/same_gpu_benchmark.json。

7.2.6 指标雷达图



图 5 三模型测试指标雷达图；mIoU 含前景与背景，EfficientNet 的第二测试 case 不同

雷达图显示 nnU-Net 与 SegResNet 的整体轮廓接近：SegResNet 在 mIoU、Dice、Accuracy 和 Recall 上略高，nnU-Net 在 Precision 上更高。EfficientNet 的 Recall 明显较低，与 `2_25_XY` 的漏分一致；但因测试 case 不同，其曲线只表示该分支结果，不能用于显著性判断。

7.2.7 三类典型切片与 Bad Case 分析

7.2.7.1 类型一：边界清晰且高度吻合

边界清晰且高度吻合 | `S_3` | `z=175` | slice Dice=0.9919

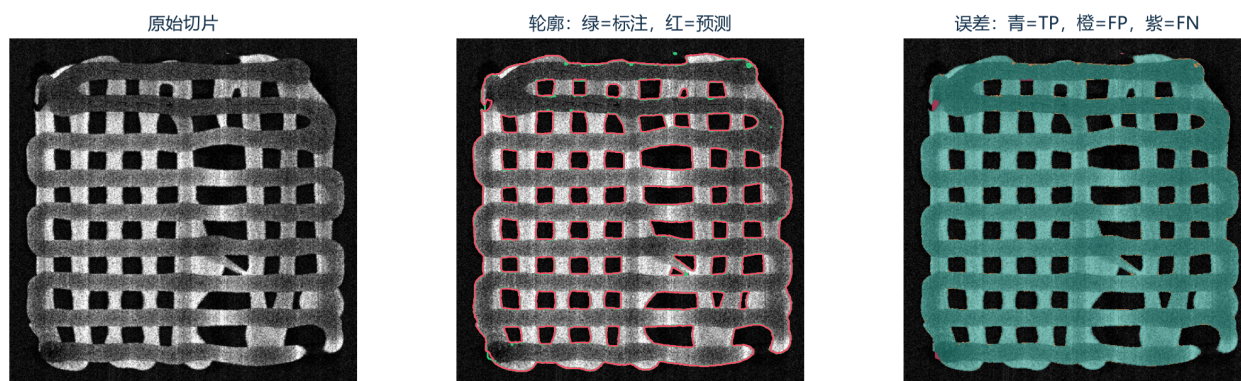


图 6 `S_3` `z=175`，slice Dice 0.9919。青色 TP 基本覆盖主体，FP/FN 极少。

该切片的目标外轮廓与内部孔洞对比清晰，预测和标注几乎重合。说明 2D nnU-Net 对训练分布内的规则网格结构可以同时恢复外边界与孔洞拓扑。少量误差主要位于薄边缘和局部高亮处。

7.2.7.2 类型二：弱边界导致漏分

弱边界导致漏分 | 2_25_XY | z=272 | slice Dice=0.6285

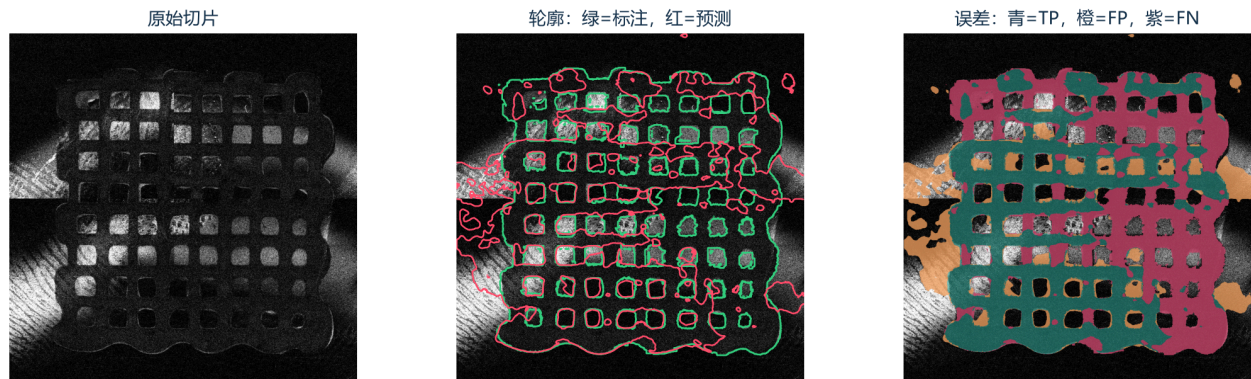


图 7 2_25_xy z=272, slice Dice 0.6285。紫色 FN 较多，表现为目标上部与内部条带漏分。

目标与背景在局部灰度接近，且上半区纹理较弱。模型倾向只保留高置信度区域，造成大片 FN；这解释了该 case 的 Recall 0.8927 低于 Precision 0.9397。改进方向包括边界/Tversky loss、困难切片重采样、多尺度上下文和 3D 邻层信息。

7.2.7.3 类型三：复杂边界导致误分

复杂边界导致误分 | 2_25_XY | z=266 | slice Dice=0.7573

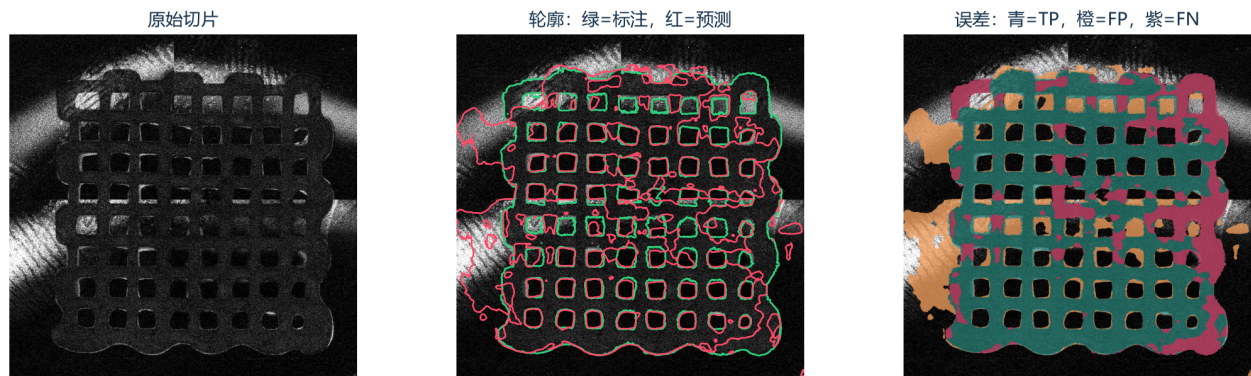


图 8 2_25_xy z=266, slice Dice 0.7573。橙色 FP 和紫色 FN 同时出现，外部高亮结构被误纳入预测。

该切片外围存在与目标强度相似的高亮结构，局部边界破碎。2D 模型无法利用相邻层的连续性判别其归属，产生 FP；内部低对比区域又产生 FN。可尝试后处理连通域、形态学约束或用 SegResNet 引入 3D 上下文，但后处理规则必须只在验证集上确定，避免测试集调参。

7.2.8 SegResNet 对照分析

边界清晰且高度吻合 | S_3 | z=175 | slice Dice=0.9901

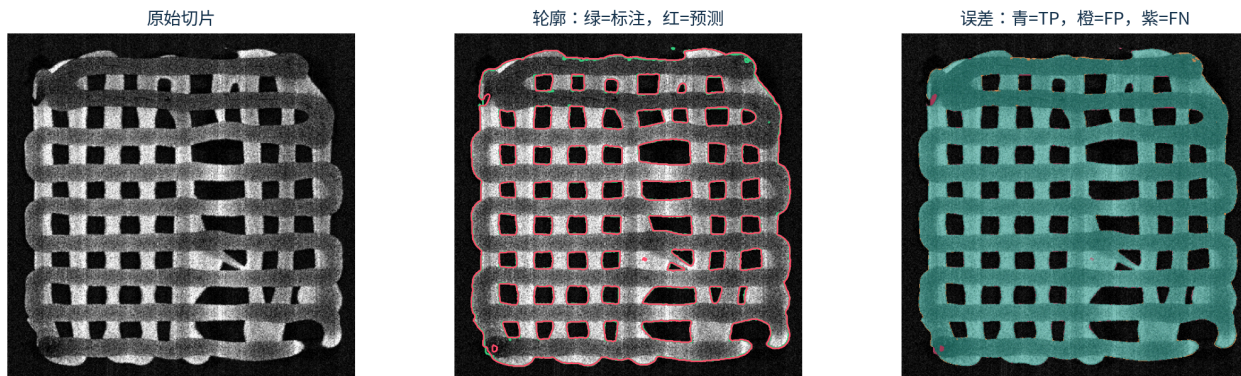


图 9 SegResNet 在 s_3 $z=175$ 的预测，slice Dice 0.9901。主体、孔洞和外边界均高度吻合。

该切片与 nnU-Net 的类型一使用同一位置，两模型均达到约 0.99 slice Dice。SegResNet 在局部边缘仍有少量 FP/FN，但 3D 上下文没有破坏规则孔洞拓扑，说明 $96 \times 96 \times 64$ patch 与滑窗融合能够恢复完整大结构。

弱边界导致漏分 | 2_25_XY | z=272 | slice Dice=0.8335

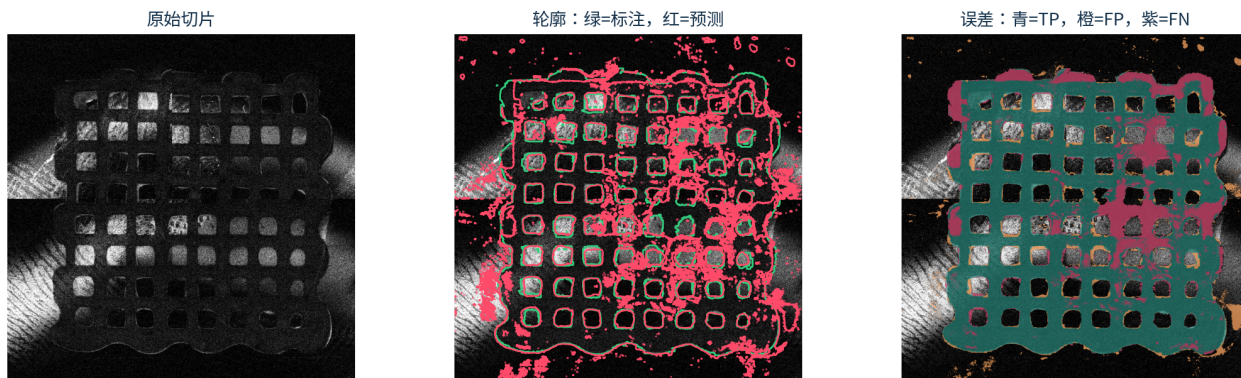


图 10 SegResNet 在 2_{25_xy} $z=272$ 的弱边界切片，slice Dice 0.8335；上半区出现较多 FN。

与 nnU-Net 在同一切片的 Dice 0.6285 相比，SegResNet 提高到 0.8335，说明相邻切片信息有助于恢复弱对比区域；但上半区仍有连续漏分，表明三维上下文不能完全替代更具代表性的训练样本和边界监督。

复杂边界导致误分 | 2_25_XY | z=233 | slice Dice=0.9299

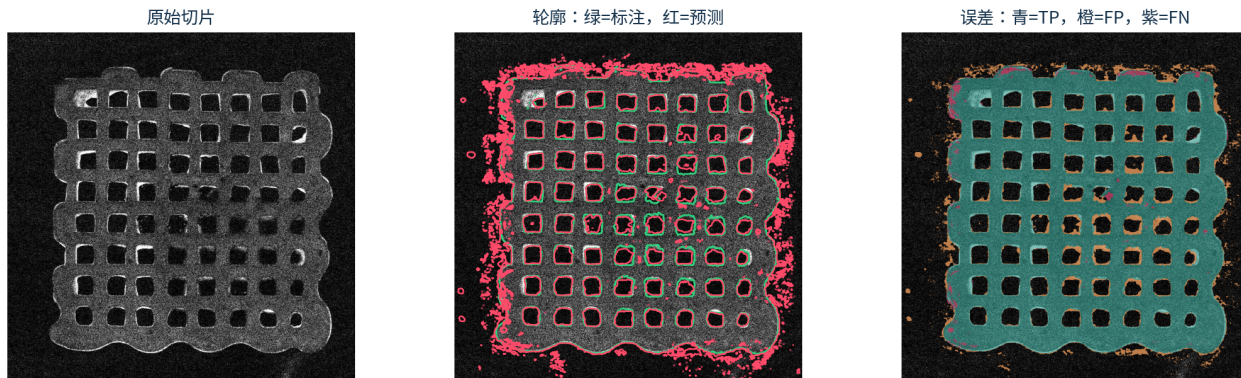


图 11 SegResNet 在 2_{25_xy} $z=233$ 的高 FP 切片，slice Dice 0.9299；外围相似纹理仍会被纳入预测。

该例总体重叠较高，但 FP 集中于目标外部且具有连续形态，说明模型把跨切片持续出现的相似纹理也解释为前景。后续可在验证集上评估最大连通域、边界损失或困难负样本采样；任何规则确定后都必须冻结，再用于测试集。

8 项目代码文件与功能介绍

项目采用 `src` layout，将课程验收入口、公共数据与评价逻辑、模型私有实现和界面层分开。根目录脚本面向助教直接运行，`src/img_seg/` 提供可复用实现，所有模型通过同一批量推理接口输出二值 mask。

8.1 课程交付入口文件

文件	功能
<code>train.py</code>	三模型统一训练入口；接收模型、配置、数据、epoch、设备与断点参数，并把任务分派给对应训练后端。
<code>test.py</code>	批量预测与评估入口；读取测试图像和可选参考 mask，统一计算 Dice、IoU、Precision、Recall 与耗时，末行输出 JSON。
<code>predict.py</code>	助教单文件推理入口；也支持目录批处理，按输入类型写出 <code>.nii.gz</code> 二值 mask 或 PNG 预览。
<code>README.md</code>	给出环境同步、权重放置以及训练、测试、单图推理和 WebUI 的可复制命令。
<code>pyproject.toml</code> / <code>uv.lock</code>	声明 Python 依赖、命令行入口并锁定环境； <code>requirements.txt</code> 仅作为 deprecated 兼容方案，不作为环境维护来源。

8.2 核心源码模块

文件或目录	功能
<code>src/img_seg/config.py</code>	读取 YAML 配置、解析项目相对路径，并为各模型提供一致的配置访问方式。
<code>src/img_seg/course_delivery.py</code>	连接根目录三个交付脚本与内部模块，完成参数兼容、临时数据配置、训练分派、预测评价和交付产物写出。
<code>data/cases.py</code> 、 <code>slices.py</code> 、 <code>splits.py</code>	发现 NIFTI case、构建 2D slice 数据集并执行固定 seed 的 case-level 划分； <code>audit_cli.py</code> 提供数据审计命令。
<code>io/nifti.py</code>	负责 NIFTI 读取、二值 mask 保存及 shape、affine/header 等空间信息保持。
<code>evaluation/metrics.py</code>	基于混淆矩阵统一实现 Dice、IoU、Precision、Recall 与 Accuracy；报告和 WebUI 共用同一评价口径。
<code>evaluation/nnunet_report.py</code> 、 <code>visualize.py</code>	汇总 nnU-Net 训练/测试日志，并生成分割叠加图和误差可视化。
<code>models/base.py</code>	定义公共模型协议与预测结果结构，隔离上层推理代码和具体网络实现。
<code>models/nnunet_v2.py</code> 、 <code>monai_segresnet.py</code> 、 <code>efficientnet_b0.py</code>	分别封装三条模型线的命令、网络构建、权重加载和单例推理逻辑。
<code>training/*_cli.py</code> 、 <code>training/monai_segresnet.py</code>	实现三个模型的训练命令；其中 SegResNet 模块包含 patch 采样、AMP 训练、验证和断点恢复。
<code>nnunet_ext_trainers/training_length.py</code>	提供课程实验使用的 200-epoch nnU-Net Trainer 扩展。
<code>inference/batch.py</code> 、 <code>inference/cli.py</code>	收集 NIFTI/普通图片输入、枚举 checkpoint、调用三模型、支持取消与进度回调、二值化输出、计算指标并生成预览。
<code>webui/app.py</code>	构建 Gradio 页面；仅调用统一 inference 接口，实现模型/权重选择、上传、批量进度、结果表、指标和预览。

8.3 配置、工具与测试

文件或目录	功能
<code>configs/data/</code>	记录数据根目录、标签规则以及 seed=42 的 train/validation/test case 清单。
<code>configs/<model>/base.yaml</code>	分别保存 nnU-Net、SegResNet 与 EfficientNet 的训练、推理和输出参数。
<code>utils/</code>	放置 NIFTI 审计、压缩、原始目录标准化与 nnU-Net 低内存预处理等独立工具。
<code>tests/</code>	覆盖数据划分、公共指标、三模型适配、批量推理、课程交付脚本和 WebUI <code>.nii.gz</code> 上传回归。

9 个人理解、实验体会与总结

9.1 对深度学习分割的理解

本项目让我更清楚地认识到，深度学习分割并不是“把网络堆得更深”就能解决的问题。模型只是在给定数据分布和损失函数下学习统计规律；如果数据划分泄漏、标签语义不一致或评价口径混乱，再高的 Dice 也没有意义。U-Net 的跳跃连接解决了语义压缩与空间定位之间的矛盾，而 nnU-Net 更进一步把大量经验转化为可复用的自动规划规则，这种系统化设计往往比孤立的结构创新更有价值。

同时，Precision、Recall 和定性误差图必须与 Dice 一起理解。2_25_xy 上 SegResNet 将 Dice 从 nnU-Net 的 0.9156 提高到 0.9342，并在弱边界切片上明显减少 FN，支持三维上下文的价值；但它在外围连续纹理处仍产生 FP。EfficientNet 参数更轻、整卷推理更快的工程潜力明确，却在较大尺寸 case 上 Recall 仅 0.8208。模型选择因此应结合漏分代价、硬件和数据规模，而不是只看单一平均分。

9.2 工程实践体会

真实训练中的主存溢出、Windows worker 失败与 WDDM 显存分配问题表明，可运行的工程链路 with 算法本身同样重要。降低 worker、按 case 推理、禁用 TTA、CPU fallback 都是为完成实验采取的工程措施；报告必须透明记录这些变化，否则速度结果无法复现。WebUI 通过统一推理接口隔离模型内部实现，也使后续接入另外两种模型时无需重写界面。

9.3 实验结论

本项目已经完成数据规范化、case-level 划分、三模型训练与整例测试、模型复杂度统计、定性误差分析和批量推理界面主体。共同测试集上，SegResNet 取得 Dice 0.9555 与 mIoU 0.9269，略高于 nnU-Net 的 0.9490 与 0.9192；nnU-Net Precision 0.9581 更高，预测更保守。EfficientNet 在其分支测试集上取得 Dice 0.9065，未达到 0.92 目标，但保持较短的 11.16 s/case 端到端时间。现有证据说明：3D 上下文有助于本任务的弱边界和跨层连续性，自配置 nnU-Net 是稳定强基线，轻量 2D 模型更适合速度优先场景。

10 参考文献

- [1] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Medical Image Computing and Computer-Assisted Intervention*, 2015, pp. 234–241. doi: [10.1007/978-3-319-24574-4_28](https://doi.org/10.1007/978-3-319-24574-4_28).
- [2] F. Isensee, P. F. Jaeger, S. A. A. Kohl, J. Petersen, and K. H. Maier-Hein, “nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation,” *Nature Methods*, vol. 18, pp. 203–211, 2021, doi: [10.1038/s41592-020-01008-z](https://doi.org/10.1038/s41592-020-01008-z).
- [3] P. A. Yushkevich *et al.*, “User-Guided 3D Active Contour Segmentation of Anatomical Structures: Significantly Improved Efficiency and Reliability,” *NeuroImage*, vol. 31, no. 3, pp. 1116–1128, 2006, doi: [10.1016/j.neuroimage.2006.01.015](https://doi.org/10.1016/j.neuroimage.2006.01.015).
- [4] L. R. Dice, “Measures of the Amount of Ecologic Association Between Species,” *Ecology*, vol. 26, no. 3, pp. 297–302, 1945, doi: [10.2307/1932409](https://doi.org/10.2307/1932409).
- [5] A. A. Taha and A. Hanbury, “Metrics for Evaluating 3D Medical Image Segmentation: Analysis, Selection, and Tool,” *BMC Medical Imaging*, vol. 15, no. 29, 2015, doi: [10.1186/s12880-015-0068-x](https://doi.org/10.1186/s12880-015-0068-x).
- [6] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, “Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning,” *Journal of Machine Learning Research*, vol. 21, no. 248, pp. 1–43, 2020.
- [7] M. J. Cardoso, W. Li, R. Brown, and others, “MONAI: An open-source framework for deep learning in healthcare,” *arXiv preprint arXiv:2211.02701*, 2022.
- [8] A. Myronenko, “3D MRI Brain Tumor Segmentation Using Autoencoder Regularization,” in *Brain-lesion: Glioma, Multiple Sclerosis, Stroke and Traumatic Brain Injuries*, 2019, pp. 311–320. doi: [10.1007/978-3-030-11726-9_28](https://doi.org/10.1007/978-3-030-11726-9_28).
- [9] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *Proceedings of the 36th International Conference on Machine Learning*, 2019, pp. 6105–6114.
- [10] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, “Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild,” *arXiv preprint arXiv:1906.02569*, 2019, doi: [10.48550/arXiv.1906.02569](https://doi.org/10.48550/arXiv.1906.02569).

11 附录：运行命令与交付清单

11.1 环境与检查

```
uv sync
uv run pytest
uv run ruff check .
uv run python utils/audit_nifti.py dataset --labels
```

11.2 nnU-Net 训练、推理与评价

```
uv run imgseg-nnunet --config configs/nnunet_v2/base.yaml prepare
uv run imgseg-nnunet --config configs/nnunet_v2/base.yaml plan
uv run imgseg-nnunet --config configs/nnunet_v2/base.yaml train --configuration 2d --fold 0
uv run imgseg-nnunet --config configs/nnunet_v2/base.yaml predict --configuration 2d --folds 0 --
checkpoint-name checkpoint_best.pth
uv run imgseg-nnunet --config configs/nnunet_v2/base.yaml evaluate --prediction-dir outputs/nnunet_v2
--reference-dir dataset/processed/nnunet_raw/Dataset501_ImgSeg/labelsTs
```

实际 CLI 的子命令参数以 `uv run imgseg-nnunet --help` 为准。

11.3 SegResNet 训练、推理与评价

```
uv run imgseg-segresnet --config configs/monai_segresnet/base.yaml inspect
uv run imgseg-segresnet --config configs/monai_segresnet/base.yaml train
uv run imgseg-segresnet --config configs/monai_segresnet/base.yaml predict --checkpoint checkpoints/
monai_segresnet/best.pt --split test --output-dir outputs/monai_segresnet/test_predictions
uv run imgseg-segresnet --config configs/monai_segresnet/base.yaml evaluate --prediction-dir outputs/
monai_segresnet/test_predictions --split test --output-json outputs/monai_segresnet/test_metrics.json
```

11.4 EfficientNet-B0 训练、推理与评价

```
uv run imgseg-efficientnet train --config configs/efficientnet_b0/base.yaml
uv run imgseg-efficientnet evaluate --config configs/efficientnet_b0/base.yaml --checkpoint
checkpoints/efficientnet_b0/20260710T152124825334Z/best.pth --split test --output-dir outputs/
efficientnet_b0 --output-json outputs/efficientnet_b0/metrics.json
uv run imgseg-predict --model efficientnet_b0 --config configs/efficientnet_b0/base.yaml --checkpoint
checkpoints/efficientnet_b0/20260710T152124825334Z/best.pth --input-dir Test --output-dir Test_Seg
```

11.5 报告构建

```
$env:NNUNET_RUN_ID = "<nnU-Net run id>"
uv run python docs/course_report/generate_assets.py --run-id $env:NNUNET_RUN_ID
New-Item -ItemType Directory -Force output/pdf | Out-Null
typst compile docs/course_report/course-report.typ output/pdf/img-seg-course-report.pdf
pdftoppm -png output/pdf/img-seg-course-report.pdf tmp/pdfs/img-seg-course-report
```